



Cálculo Numérico

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

1

Introdução ao estudo da matemática numérica e aritmética de máquina

Resumo

Python é uma linguagem de programação poderosa de uso geral. Pode ser usada de forma interativa, permitindo um desenvolvimento muito rápido. Possui muitas ferramentas de computação científica poderosas, tornando-a ideal para matemática aplicada e computacional. Nesta introdução apresentamos a sintaxe do Python, tipos de dados, funções e ferramentas de controle de fluxo. Esses princípios básicos do Python são uma parte essencial de quase todos os problemas que iremos resolver ao longo da disciplina.

“A mente que se abre a uma nova ideia jamais volta ao seu tamanho original.”

Albert Einstein

Vamos começar

Python é uma linguagem de programação multiparadigma, com forte suporte à **Programação Orientada a Objetos (POO)**. Ou seja, Python funciona conforme os princípios clássicos de POO, onde:

- Um **objeto** é uma instância concreta de uma classe (blueprint).
- Cada objeto possui **atributos**, que armazenam dados ou estado.
- Um objeto também possui **métodos**, que definem comportamentos e operações acessíveis ao objeto [1].

Python tem ganhado relevância acelerada como ferramenta para computação científica, análise de dados, automação e prototipagem rápida [2].

Arquivos com códigos Python devem ser salvos com a extensão `.py`. Para iniciantes, recomenda-se o uso de editores de texto simples (como VS Code, Gedit, Notepad++), que minimizam distrações e complexidades desnecessárias. Embora IDEs completas possam ajudar, ambientes minimalistas favorecem a compreensão dos fundamentos da linguagem. A seguir, um exemplo de arquivo Python básico:

Código: `arquivo.py`

```
1 # Isso eh um comentario.
2 """
3 Isto eh um arquivo de cabecalho.
4 O cabecalho contem informacoes basicas sobre o arquivo.
5 """
6 if __name__ == "__main__":
7     pass                                # 'pass' eh um espaco reservado temporario.
```

O caractere `#` cria um *comentário* de linha única. Os comentários são ignorados pelo intérpretor Python e servem como anotações para o código-fonte que os acompanha. Um par de três aspas (simples ou duplas) cria uma *string* literal de múltiplas linhas, que também podem ser usadas como um comentário de múltiplas linhas. Porém, seu principal objetivo nos códigos é servir como uma documentação automática a constar, por exemplo, em arquivos *README*.

Problema

- 1.1 Abra um arquivo com o nome `python_intro.py` (ou crie o arquivo no editor de texto). Adicione as seguintes informações ao cabeçalho na parte superior.

Código: `python_intro.py`

```
1 if __name__ == "__main__":
2     print("Ola mundo!") # Identacao com quatro espacos (Nao foi usado tab).
```

Tenha certeza de que o arquivo foi salvo. Abra um terminal de *prompt* (*Terminal* em Linux ou Mac e *Command Prompt* ou *GitBash* em Windows) e navegue até o diretório onde o arquivo foi salvo. Use o comando `ls` (or `DIR` em Windows) para listar os arquivos e pastas do diretório corrente, `pwd` (`cd`, em Windows) para imprimir o diretório em que está trabalhando, e `cd` para mudar entre os diretórios.

```
$ pwd          # Print the working directory.
/Users/Guest
$ ls           # List the files and folders here.
Desktop      Documents    Downloads    Pictures     Music
$ cd Documents # Navigate to a different folder.
$ pwd
/Users/Guest/Documents
$ ls          # Check to see that the file is here.
python_intro.py
```

Tendo encontrado o arquivo Python, ele pode ser executado através do comando:

```
$ python python_intro.py
```

Se `Olá Mundo!` for exibido na sua tela, a execução do programa ocorreu como esperado!

NOTA

A expressão `if __name__ == "__main__"` é extremamente útil para criar funções de teste e depurar seu código. Para usar `if __name__ == "__main__"` para testar funções declaradas, **deve ser colocado no final do seu arquivo** com o código que você deseja executar diretamente a seguir. Se você tentar executar esse bloco de código no início do arquivo com funções que são declaradas posteriormente, você receberá um erro de ter funções indefinidas.

IPython

O Python pode ser utilizado de forma interativa por meio de diferentes interfaces. Uma das mais populares e versáteis é o *IPython*¹, que consiste em um interpretador interativo baseado em console, com recursos avançados que superam os do interpretador padrão da linguagem.

No IPython, as entradas de código são geralmente precedidas por colchetes angulares numerados (veja, por exemplo, In Pérez et. al. (2007) [1]), indicando que o comando será executado imediatamente. Para executar um *script* diretamente no ambiente IPython, utiliza-se o comando `%run`, conforme ilustrado abaixo:

```

1 $ ipython                                # Iniciar o IPython.
2
3 In [1]: print("Isso eh simplesmente IPython!") # Executa o comando.
4 Isso eh simplesmente IPython!
5
6 In [2]: %run python_intro.py              # Executa um script Python.
7 Ola mundo!
```

Uma das principais vantagens do IPython reside no suporte à introspecção de objetos — funcionalidade que permite visualizar, de forma prática, os atributos e métodos associados a objetos da linguagem, contribuindo para o processo de aprendizagem e depuração de código [3]. Além disso, o ambiente oferece a função integrada `help()`, que facilita o acesso à documentação e promove uma experiência interativa para o usuário.

```

1 # Uma lista eh uma estrutura de dados fundamental em Python.
2 # Para visualizar os metodos associados a uma lista, digite o nome do objeto seguido
   de um ponto e pressione 'tab'.
3 In [1]: list.    # Pressione 'tab'.
4 append()  clear()  copy()    count()  extend()
5 index()   insert() pop()     remove() reverse() sort()
6
7 # Para acessar a documentacao de um metodo especifico, utilize o caractere '?'.
8 In [2]: list.append?
9 Signature: list.append(self, object, /)
10 Docstring: Append object to the end of the list.
11 Type:      method_descriptor
```

¹Disponível em: <https://ipython.org/>. Acesso em: 06 ago. 2025.

```
12 # Alternativamente, utilize a funcao 'help()' para acessar ajuda mais extensa:
13 In [3]: help()
14 help> list
15 Help on class list in module builtins:
16 class list(object)
17 | list(iterable=(), /)
18 | ...
19 # Pressione 'q' para sair.
20 help> quit
```

NOTA

Utilizar o IPython em paralelo a um editor de texto é uma prática recomendada, especialmente durante o aprendizado. Esse ambiente permite testar rapidamente a sintaxe e pequenos blocos de código, o que favorece a identificação precoce de erros e torna o processo de desenvolvimento mais eficiente.

Em caso de dúvidas, recorrer a fontes de apoio como o site stackoverflow.com é altamente indicado, dada a sua ampla comunidade e base de conhecimento consolidada.

Por fim, vale lembrar que a melhor forma de aprender uma linguagem de programação é **escrevendo código**. Portanto, pratique os exemplos apresentados ao longo deste material, executando-os diretamente no console do IPython. Evite, sempre que possível, copiar e colar os comandos — ao digitar, você treina não apenas a lógica, mas também a sintaxe da linguagem.

Python Básico

Aritmética

Python pode ser usado como uma calculadora, com os operadores básicos regulares: +, -, *, and /. Use ** para exponenciação e \% para divisão modular.

```
1 >>> 3**2 + 2*5                # Python obedece ordem de operacoes.
2     19
3
4 >>> 13 % 3                    # O operador % calcula o
5     1                          # resto da divisao: 13 = (3*4) + 1.
```

Em muitos interpretadores Python, o caracter *underscore* (sublinhado) `_` é uma variável com o valor da saída do comando anterior, como o botão ANS em muitas calculadoras.

```
1 >>> 12 * 3
2     36
3 >>> _ / 4
4     9.0
```

Os colchetes angulares, de comparação de dados, < e >, atuam como esperado. O operador == verifica a igualdade numérica e os operadores <= e >= correspondem às desigualdades ≤ (menor ou igual) e ≥ (maior ou igual), respectivamente. Para conectar multiplas expressões booleanas utilize os operadores and, or e not².

```
1 >>> 3 > 2.99
2     <<True>>
3 >>> 1.0 <= 1 or 2 > 3
4     <<True>>
5 >>> 7 == 7 and not 4 < 4
6     <<True>>
7 >>> True and True and True and True and True and False
8     <<False>>
9 >>> False or False or False or False or False or True
```

²Em muitas outras linguagens de programação os operadores and, or, e not são escritos como &&, || e !, respectivamente. A convenção de Python é muito mais objetiva, não requerendo a adição de marcadores como parênteses, por exemplo.

```

10     <<True>>
11 >>> True or not True
12     <<True>>

```

Variaveis

Um sinal de igualdade **único** = atribui um ou mais valores (à direita) a um ou mais nomes de variáveis (à esquerda). Um **duplo** sinal de igualdade == é um operador de comparação que retorna True ou False, como no bloco de código anterior.

Ao contrário de muitas linguagens de programação, Python não exige que o tipo de dados de uma variável seja especificado na inicialização. Por causa disso, Python é chamado de linguagem *digitada dinamicamente*.

```

1 >>> x = 12                                # Initialize x with the integer 12.
2 >>> y = 2 * 6                             # Initialize y with the integer 2*6 = 12.
3 >>> x == y                               # Compare the two variable values.
4     True
5 >>> x, y = 2, 4                          # Give both x and y new values in one line.
6 >>> x == y
7     False

```

Funções

Para definir uma função, use a palavra-chave def seguida pelo nome da função, uma lista de parâmetros entre parênteses e dois pontos. Em seguida, recue o corpo da função usando exatamente **quatro** espaços.

```

1 >>> def add(x, y):
2     ...     return x + y                # Indent with four spaces.

```

NOTA

Diferentemente de muitas linguagens de programação que utilizam chaves ({}), para delimitar blocos de código, o **Python utiliza a indentação (reco com**

espaços em branco) como parte da sua sintaxe. Esse design torna o código mais legível e menos propenso a erros visuais.

No entanto, Python é sensível à forma como essa indentação é aplicada. **Misturar espaços e tabulações pode confundir o interpretador e causar erros difíceis de depurar.** A própria documentação oficial recomenda o uso consistente de **quatro espaços por nível de indentação**, evitando o uso de tabulações [4].

A maioria dos editores modernos permite configurar a tecla Tab para inserir automaticamente quatro espaços (conhecido como *soft tabs*). Para manter consistência e compatibilidade, **nunca use tabulações — sempre utilize espaços.**

Funções são definidas com *parâmetros* e chamadas com *argumentos*, embora os termos sejam frequentemente usados de forma intercambiável. Abaixo, `width` e `height` são parâmetros para a função `area()`. Os vares 2 e 5 são os argumentos passados ao chamar a função.

```
1 >>> def area(width, height):           # Define the function.
2 ...     return width * height
3 ...
4 >>> area(2, 5)                         # Call the function.
5     10
```

As funções Python também podem retornar vários valores.

```
1 >>> def arithmetic(a, b):
2 ...     return a - b, a * b           # Separate return values with commas.
3 ...
4 >>> x, y = arithmetic(5, 2)           # Unpack the returns into two variables.
5 >>> print(x, y)
6     3 10
```

A palavra-chave `lambda` é um atalho para criar funções de uma linha. Por exemplo, os polinômios $f(x) = 6x^3 + 4x^2 - x + 3$ e $g(x, y, z) = x + y^2 - z^3$ podem ser definidos como funções em uma linha cada.

```
1 # Define the polynomials the usual way using 'def'.
2 >>> def f(x):
3 ...     return 6*x**3 + 4*x**2 - x + 3
4 >>> def g(x, y, z):
```

```
5 ...     return x + y**2 - z**3
6 # Equivalently, define the polynomials quickly using 'lambda'.
7 >>> f = lambda x: 6*x**3 + 4*x**2 - x + 3
8 >>> g = lambda x, y, z: x + y**2 - z**3
```

NOTA

A documentação é importante em todas as linguagens de programação. Cada função deve ter um *docstring* — uma *string* literal entre aspas triplas, logo abaixo da declaração da função — que descreve o propósito da função, as entradas esperadas e valores de retorno, bem como quaisquer outras notas que sejam importantes para o usuário. Documentos curtos são aceitáveis para funções muito simples, porém funções mais complicadas requerem explicações cuidadosas e detalhadas.

```
1 >>> def add(x, y):
2 ...     """Return the sum of the two inputs."""
3 ...     return x + y
4
5 >>> def area(width, height):
6 ...     """Return the area of the rectangle with the specified width
7 ...     and height.
8 ...     """
9 ...     return width * height
10 ...
11 >>> def arithmetic(a, b):
12 ...     """Return the difference and the product of the two inputs."""
13 ...     return a - b, a * b
```

As funções Lambda não podem ter docstrings personalizados, portanto a palavra-chave `lambda` deve ser usada apenas como um atalho para funções muito simples ou intuitivas que não precisam de rotulagem adicional.

A instrução `return` encerra instantaneamente a chamada da função e passa o valor de retorno para o chamador da função. No entanto, as funções não precisam ter uma instrução `return`. Uma função sem uma instrução `return` retorna implicitamente a constante Python `None`, que é semelhante ao valor especial `null` de muitas outras linguagens. Chamar `print()` no final

de uma função **não** faz com que a função retorne quaisquer valores.

```

1 >>> def oops(i):
2     ...     """Increment i (but forget to return anything)."""
3     ...     print(i + 1)
4     ...
5 >>> def increment(i):
6     ...     """Increment i."""
7     ...     return i + 1
8     ...
9 >>> x = oops(1999)                # x contains 'None' since oops()
10 2000                            # doesn't have a return statement.
11 >>> y = increment(1999)          # However, y contains a value.
12 >>> print(x, y)
13 None 2000

```

Se você tiver alguma intenção de usar os resultados de uma função, use uma instrução `return`.

Também é possível especificar *valores padrão* para os parâmetros de uma função. No exemplo a seguir, a função `pad()` possui três parâmetros e o valor de `c` é padronizado como 0. Se não for especificado na chamada da função, a variável `c` conterá o valor 0 quando a função for executada.

```

1 >>> def pad(a, b, c=0):
2     ...     """Print the arguments, plus a zero if c is not specified."""
3     ...     print(a, b, c)
4     ...
5 >>> pad(1, 2, 3)                  # Specify each parameter.
6 1 2 3
7 >>> pad(1, 2)                    # Specify only non-default parameters.
8 1 2 0

```

É importante observar que os argumentos posicionais devem preceder os argumentos nomeados em uma chamada de função. Além disso, os parâmetro sem valores padrão devem preceder os parâmetros com valores padrão em uma definição de função. Por exemplo, `a` e `b` devem vir antes de `c` na definição da função `pad()`. Examine os blocos de código a seguir que demonstram como argumentos posicionais e nomeados são usados para chamar uma função.

```
1 # Try defining pad() with a named argument before a positional argument.
2 >>> def pad(c=0, a, b):
3 ...     print(a, b, c)
4 ...
5 <<SyntaxError: non-default argument follows default argument>>
6 # Correctly define pad() with the named argument after positional arguments.
7 >>> def pad(a, b, c=0):
8 ...     """Print the arguments, plus a zero if c is not specified."""
9 ...     print(a, b, c)
10 ...
11
12 # Call pad() with 3 positional arguments.
13 >>> pad(2, 4, 6)
14 2 4 6
15 # Call pad() with 3 named arguments. Note the change in order.
16 >>> pad(b=3, c=5, a=7)
17 7 3 5
18
19 # Call pad() with 2 named arguments, excluding c.
20 >>> pad(b=1, a=2)
21 2 1 0
22
23 # Call pad() with 1 positional argument and 2 named arguments.
24 >>> pad(1, c=2, b=3)
25 1 3 2
```

NOTA

Nas versões anteriores do Python, `print()` era uma *instrução* (como `return`), não uma função, e poderia, portanto, ser executada sem parênteses. No entanto, faltavam argumentos de palavras-chave como `sep` e `end`. Se você estiver usando Python 2.7, inclua a seguinte linha no topo do arquivo para transformar a instrução `print` na nova função `print()`.

```
1 >>> from __future__ import print_function
```

Tipos e Estruturas de Dados

Tipos Numéricos

Python tem quatro tipos de dados numéricos: `int`, `long`, `float`, and `complex`. Cada um deles armazena um tipo diferente de número. Each stores a different kind of number.

A função integrada `type()` identifica o tipo de dados de um objeto.

```
1 >>> type(3)                                # Numbers without periods are integers.
2 <<int>>
3 >>> type(3.0)                              # Floats have periods (3. is also a float).
4 <<float>>
```

Python possui dois tipos de divisão: inteiro e flutuante. O operador `/` realiza a divisão flutuante (divisão fracionária verdadeira), e o operador `//` realiza a divisão inteira, que arredonda o resultado para o próximo número inteiro. Se ambos os operandos de `//` forem números inteiros, o resultado será um `int`. Se um ou ambos os operandos forem flutuantes, o resultado será um `float`. A divisão regular com `/` sempre retorna um `float`.

```
1 >>> 15 / 4                                # Float division performs as expected.
2      3.75
3 >>> 15 // 4                              # Integer division rounds the result down.
4      3
5 >>> 15. // 4
6      3.0
```

NOTA

Nas versões anteriores do Python, usar `/` com dois inteiros realizava a divisão de inteiros, mesmo nos casos em que a divisão não era par. Isso pode resultar em alguns erros incrivelmente sutis e frustrantes. Se você estiver usando Python 2.7, sempre inclua um `.` nos operandos ou converta pelo menos um como `float` quando desejar divisão `float`.

```
1 # PYTHON 2.7
2 >>> 15 / 4                                # The answer should be 3.75, but the
```

```
3      3                                # interpreter does integer division!
4
5 >>> 15. / float(4)                    # 15. and float(4) are both floats, so
6      3.75                            # the interpreter does float division.
```

Como alternativa, incluir a linha a seguir na parte superior do arquivo redefine os operadores / e // para que sejam tratados da mesma forma que no Python 3.

```
1 >>> from __future__ import division
```

Python também oferece suporte a cálculos de números complexos, emparelhando dois números como partes reais e imaginárias. Use a letra *j*, e não *i*, para a parte imaginária.

```
1 >>> x = complex(2, 3)                # Create a complex number this way...
2 >>> y = 4 + 5j                       # ...or this way, using j (not i).
3 >>> x.real                           # Access the real part of x.
4      2.0
5 >>> y.imag                           # Access the imaginary part of y.
6      5.0
```

Strings

Em Python, *strings* são criadas com aspas simples ou duplas. Para concatenar duas ou mais *strings*, use o operador + entre variáveis de *strings* ou literais.

```
1 >>> str1 = "Ola"
2 >>> str2 = 'mundo'
3 >>> my_string = str1 + " " + str2 + '!'
4 >>> my_string
5 <<'Hello world! '>>
```

Partes de uma *string* podem ser acessadas usando *slicing*, indicado por colchetes []. A sintaxe de fatiamento é [start:stop:step]. Os parâmetros start e stop são padronizados para o início e o fim da string, respectivamente. O parâmetro step tem como padrão 1.

```
1 >>> my_string = "Hello world!"
```

```

2 >>> my_string[4]          # Indexing begins at 0.
3     <<'o'>>
4 >>> my_string[-1]         # Negative indices count backward from the end.
5     <<'!'>>
6
7 # Slice from the 0th to the 5th character (not including the 5th character).
8 >>> my_string[:5]
9     <<'Hello'>>
10
11 # Slice from the 6th character to the end.
12 >>> my_string[6:]
13     <<'world! '>>
14
15 # Slice from the 3rd to the 8th character (not including the 8th character).
16 >>> my_string[3:8]
17     <<'lo wo'>>
18
19 # Get every other character in the string.
20 >>> my_string[::2]
21     <<'Hlowrd'>>

```

Listas

Uma lista (list) em Python é criada quando utilizamos colchetes (quadrados, []) e separamos seus valores por vírgulas. Os elementos de uma lista não precisam ser do mesmo tipo. O acesso às entradas em uma lista ocorre por meio de operações de indexação ou fatiamento.

```

1 >>> my_list = ["Ola", 93.8, "mundo", 10]
2 >>> my_list[0]
3     <<'Ola'>>
4 >>> my_list[-2]
5     <<'mundo'>>
6 >>> my_list[:2]
7     <<['Ola', 93.8]>>

```

As listas aceitam os seguintes métodos: `append()`, `insert()`, `remove()`, `extend()`, e `pop()`.

```
1 >>> my_list = [1, 2]           # Create a simple list of two integers.
2 >>> my_list.append(4)          # Append the integer 4 to the end.
3 >>> my_list.insert(2, 3)       # Insert 3 at location 2.
4 >>> my_list
5     [1, 2, 3, 4]
6 >>> my_list.remove(3)         # Remove 3 from the list.
7 >>> my_list.pop()             # Remove (and return) the last entry.
8     4
9 >>> my_list
10    [1, 2]
11 >>> my_list.extend([5, 6])    # Append multiple values at once.
12 >>> my_list
13    [1, 2, 5, 6]
```

O fatiamento também é muito útil para substituir valores em uma lista.

```
1 >>> my_list = [10, 20, 30, 40, 50]
2 >>> my_list[0] = -1
3 >>> my_list[3:] = [8, 9]
4 >>> print(my_list)
5     [-1, 20, 30, 8, 9]
```

O operador `in` verifica rapidamente se um determinado valor está em uma lista (ou outro iterável, incluindo strings).

```
1 >>> my_list = [1, 2, 3, 4, 5]
2 >>> 2 in my_list
3     <<True>>
4 >>> 6 in my_list
5     <<False>>
6 >>> 'a' in "xylophone"       # 'in' also works on strings.
7     <<False>>
```

 Lista de Exercícios

1.1 Converter as seguintes expressões para o interpretador Python:

(a) $10 + 20 \times 30$

(b) $4^2 \div 30$

(c) $(9^4 + 2) \times 6 - 1$

1.2 No interpretador Python (ou em um arquivo.py) resolva a expressão: $10\%3 \times 10^2 + 1 - 10 \times 4/2$. Resolva também usando apenas lápis e papel. O resultado a ser encontrado é 81.0

1.3 Faça um programa que resolva a soma de três variáveis e imprima o resultado na tela.

1.4 Considerando um salário de R\$ 750.00, determine numericamente um aumento de 15%.

1.5 Escreva um programa que, a partir da entradas dos valores de dias, horas, minutos e segundos, calcule o tempo total em segundos.

1.6 Escreva um programa que pergunte três números para o usuário e que devolva como resposta qual o maior deles.

1.7 Escreva um programa em Python que verifique se um número é primo.

1.8 O volume de uma esfera com raio r é $V = \frac{4}{3}\pi r^3$. Em seu arquivo Python do problema **1.1**, defina uma função chamada `sphere_volume()` que aceita um único parâmetro r . Retorne o volume da esfera de raio r , usando 3,14159 como aproximação para π (por enquanto). Escreva também uma documentação apropriada para sua função.

Para testar sua função, chame-a na cláusula `if __name__ == "__main__"` e imprima o valor retornado. Execute seu arquivo para ver se sua resposta é o que você espera.

1.9 A função integrada `print()` possui os argumentos de palavras-chave úteis `sep` e `end`. Ele aceita qualquer número de argumentos posicionais e os imprime com `sep`

inserido **entre** valores (padrão para um **espaço**), então imprime **end** (padrão para o caractere de nova linha '**\n**').

Escreva uma função chamada `isolate()` que aceite **cinco** argumentos. A função deve imprimir os **três** primeiros argumentos separados por **5 espaços** e depois imprimir os **dois** últimos argumentos com **um único** espaço separando os três últimos argumentos. Por exemplo,

```
1 >>> isolate(1, 2, 3, 4, 5)
2 1      2      3 4 5
```

1.10 Escreva duas novas funções, chamadas `first_half()` e `backward()`.

- (a) `first_half()` deve aceitar um parâmetro e retornar a primeira metade dele, excluindo o caractere do meio se houver um número ímpar de caracteres.
(Dica: a função interna `len()` retorna o comprimento da entrada.)
- (b) A função `backward()` deve aceitar um parâmetro e inverter a ordem de seus caracteres usando fatiamento e, em seguida, retornar o parâmetro invertido.
(Dica: o parâmetro `step` usado no fatiamento pode ser negativo.)

Use o IPython para testar rapidamente a sintaxe de cada função.

Referências Bibliográficas

- [1] FOUNDATION, P. S. *PEP257 – Docstring Conventions*. 2001. <https://peps.python.org/pep-0257/>.
- [2] FOUNDATION, P. S. *Builtin Module `__main__` — Python Documentation*. 2025. https://docs.python.org/3/library/__main__.html.
- [3] PÉREZ, F.; GRANGER, B. E. Ipython: a system for interactive scientific computing. *Computing in Science & Engineering*, IEEE, v. 9, n. 3, p. 21–29, 2007.
- [4] ROSSUM, G. V.; WARSAW, B.; COGHLAN, N. *PEP 8 – Style Guide for Python Code*. 2001. <https://peps.python.org/pep-0008/>. Python Enhancement Proposal.